

An Empirical Study of Energy Efficiency Monitoring for Container-Based Data Exchange Systems

Alexandros Koufakis

Universiteit van Amsterdam
Amsterdam, The Netherlands
a.m.koufakis@uva.nl

Tom van Engers

Universiteit van Amsterdam
Amsterdam, The Netherlands
T.M.vanEngers@uva.nl

Collin Poetoehena

Universiteit van Amsterdam
Amsterdam, The Netherlands

Ana Opreescu

Universiteit van Amsterdam
Amsterdam, The Netherlands
a.m.oprescu@uva.nl

Abstract

Optimizing energy efficiency in software systems has become an increasingly critical concern. The ICT sector contributes significantly to global greenhouse gas emissions, with cloud computing and microservice-based architectures playing a substantial role in this energy demand. Moreover, with the advent of various data exchange technologies, data-centric multi-domain applications are growing. We address this challenge by investigating energy efficiency optimizations for data exchange archetypes within policy-aware dynamically adaptive microservice-based middleware.

We introduce an energy consumption monitoring pipeline applicable to microservice-based policy-aware middleware designed to support data exchanges, such as DYNAMOS (Dynamically Adaptive Microservice-based OS). The monitoring pipeline is integrated in the system's architecture and utilizes monitoring tools to retrieve energy consumption data and statistical tools to identify which stages of the execution contributed the most to energy consumption. Using this analysis, we design and implement two optimizations that demonstrate reductions in energy usage. Both optimisations are deployed in both centralized and decentralized setups.

CCS Concepts

• **General and reference** → *Metrics*; **Empirical studies**; • **Hardware** → **Impact on the environment**.

Keywords

Energy efficiency, Distributed systems, Data exchange archetypes

ACM Reference Format:

Alexandros Koufakis, Collin Poetoehena, Tom van Engers, and Ana Opreescu. 2026. An Empirical Study of Energy Efficiency Monitoring for Container-Based Data Exchange Systems. In *10th International Workshop on Green and Sustainable Software (GREENS '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3786148.3788628>

ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only. Request permissions from owner/author(s). *GREENS '26*, Rio de Janeiro, Brazil

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2381-0/2026/04
<https://doi.org/10.1145/3786148.3788628>

1 Introduction

Since cloud computing and microservice-based architectures are playing a substantial role in the world's energy demand and hence to global greenhouse gas emissions, we set out to investigate energy efficient data exchange systems.

The importance of energy efficiency in ICT has increased with the fast expansion of data centers and cloud computing [12]. In 2021 Freitag et al. [13] suggested that the ICT sector's share of global greenhouse gas (GHG) emissions could reach 2.1% – 3.9% and is projected to grow in the future. Therefore, addressing energy efficiency in ICT is not only a technical challenge but also a societal imperative. Among various ICT architectures, microservice-based systems have gained significant traction due to their scalability and flexibility. However, these systems also introduce additional overhead and complexity that can impact energy consumption. Therefore, improving the energy efficiency of such systems is a key step toward sustainable software [11, 21].

Balanza-Martinez et al. [7] emphasize that algorithms and strategies to improve energy efficiency require deeper investigation, urging researchers and practitioners to prioritize this area to advance global sustainability goals. A key gap highlighted in literature is the lack of empirical studies on energy efficiency optimizations, particularly the scarcity of actionable, implementable solutions.

DYNAMOS¹ (Dynamically Adaptive Microservice-based OS) represents a promising platform to address this gap. DYNAMOS is a framework that enables secure data exchange and includes a methodology to dynamically select and compose data exchange archetypes. While DYNAMOS already supports adaptation based on various policies, its potential to optimize energy consumption remains largely unexplored. Its ability to orchestrate archetypes dynamically creates new opportunities to integrate energy-efficient strategies directly into system behavior, making it an ideal testbed for evaluating energy efficiency optimizations.

We investigate these opportunities by applying an energy efficiency monitoring pipeline built on Kubernetes-based monitoring tools. Inspired by the work of Floroiu et al. [12], this pipeline begins with energy consumption measurements and applies an anomaly detection algorithm to identify deviations or abnormalities. A subsequent Root Cause Analysis (RCA) phase determines the underlying factors contributing to changes in the energy consumption. Based

¹github.com/Jorrit05/DYNAMOS

on these insights, we demonstrate how the findings can guide the optimization of data exchange operations. Finally, the optimizations are evaluated against their original counterparts in both centralized and distributed environments, the latter deployed on FABRIC².

In this paper, we make three main contributions:

- (1) We investigate energy consumption monitoring on distributed systems and its application to the DYNAMOS [25] system.
- (2) We demonstrate how energy consumption monitoring and diagnostics can identify avenues for improvement and we develop two optimizations for data exchange archetypes.
- (3) We provide an extensive evaluation on local and distributed deployment on the FABRIC testbed.

Section 2 provides relevant background on the methodologies and tools used in this work. Section 3 details the implementation of energy measurement and root cause analysis. Section 4 describes how the monitoring pipeline guided the implementation of two optimizations to the data exchange archetypes. Section 5 presents the experimental setup and the evaluation of the optimizations. Finally, Section 6 summarizes the findings and concludes this work.

2 Background

In this section we introduce the core concepts used in this work: green telemetry, DYNAMOS and Data Exchange Archetypes.

2.1 Green Computing and Telemetry

Green computing refers to the environmentally conscious design, use, and disposal of computing resources, with the goal of minimizing energy consumption, reducing e-waste, and lowering the overall environmental footprint of IT systems. As Information Communication Technology (ICT) has expanded exponentially, energy use and electronic waste have grown in parallel, making sustainability a pressing concern in both research and practice [1].

Core practices of green computing include designing energy-efficient hardware and software, recycling and repurposing outdated components, and minimizing resource use during manufacturing and operation. Recent developments also highlight advances in cloud computing efficiency, better hardware utilization, and sustainable management of Internet of Things (IoT) devices, which continue to grow rapidly in both scope and scale [1].

A key dimension of green computing is the role of software in driving sustainability. Here, green software can be divided into two categories [7]: (i) green in software, focuses on optimizing software to reduce its own energy consumption; (ii) green by software, refers to software that facilitates energy-efficient practices in other domains or systems. This distinction is crucial, as it highlights that software can either directly shrink its environmental footprint or act as a facilitator for broader sustainability.

In the context of this work, green computing principles are operationalized through a focus on improving energy efficiency within DYNAMOS. By implementing and empirically validating energy efficient strategies, this research contributes to the broader objectives of green computing. Specifically, we focus on “green in software”, targeting internal optimizations within DYNAMOS to achieve tangible reductions in energy consumption.

²FABRIC is Adaptive Programmable Research Infrastructure for Computer Science and Science Applications; portal.fabric-testbed.net/

In order to effectively adopt green computing principles, it is necessary to understand the energy consumption characteristics of existing software. There are multiple energy monitoring tools that can enable deeper understanding of the energy consumption, for example, Floroiu et al. [12] utilize Scaphandre for energy monitoring. Currently, the two most widely used energy monitoring tools are Scaphandre and Kepler³.

Scaphandre is a monitoring agent specifically designed for tracking energy consumption metrics⁴. It facilitates the measurement and understanding of energy consumption patterns in technological services. Scaphandre integrates with Prometheus⁵, enabling real-time monitoring. Kepler (Kubernetes-based Efficient Power Level Exporter) leverages eBPF (extended Berkeley Packet Filter) technology⁶ to track per-container energy consumption efficiently. Kepler probes CPU performance counters and kernel tracepoints, using these metrics along with machine learning models to estimate energy consumption accurately. Kepler is designed to integrate with Kubernetes, and its measurements are aligned with the Greenhouse Gas (GHG) protocol [4, 5].

2.2 DYNAMOS

We describe DYNAMOS and the motivation to select it as a suitable case study for a complex microservice-based system. First, it uses Kubernetes⁷, a widely adopted open-source platform for containerized deployment. Moreover, its operations are organized into a taxonomy of archetypes, which makes it more likely that the results can be extrapolated to other use cases falling under the same archetypes. Finally, DYNAMOS is designed to work with distributed collaborators, which is common in real-world scenarios.

DYNAMOS is inspired by the architecture of Digital Data-sharing Marketplaces (DDMs), *i.e.*, platforms where multiple parties securely exchange data under strict policy control, ensuring sovereignty over data assets. A key concept of DDMs is the use of archetypes [29] — common data-sharing patterns that define how data is exchanged, including who provides the data, who processes it, and where results are delivered, all according to agreed-upon policies [22, 24]. For instance, one archetype may involve data being processed by a trusted third party before reaching the end user, while another may involve computation occurring directly at the data source. There are 11 archetypes commonly referenced in the literature (Figure 1), two of which are already implemented in DYNAMOS.

A central feature of DYNAMOS is its ability to dynamically select data exchange archetypes in response to policies, user inputs, and system events, offering significant flexibility and self-adaptivity [24]. In practice, DYNAMOS uses extendable algorithms to create dynamic microservice chains tailored to each request. DYNAMOS may choose to use the Compute to Data (CtD) instead of the Data via Trusted Third Party (DvTTP) archetype according to the selection mechanism. Currently, DYNAMOS selects the archetypes based on a scoring system, but it is designed so that it can be easily extended. For example, the archetype selection mechanism can be adapted to optimize for lower computational cost of the archetypes.

³github.blog/open-source/social-impact/the-10-best-tools-to-green-your-software

⁴hubblo-org.github.io/scaphandre-documentation/references/metrics.html

⁵prometheus.io/docs/introduction/overview/

⁶ebpf.io/what-is-ebpf/

⁷kubernetes.io/

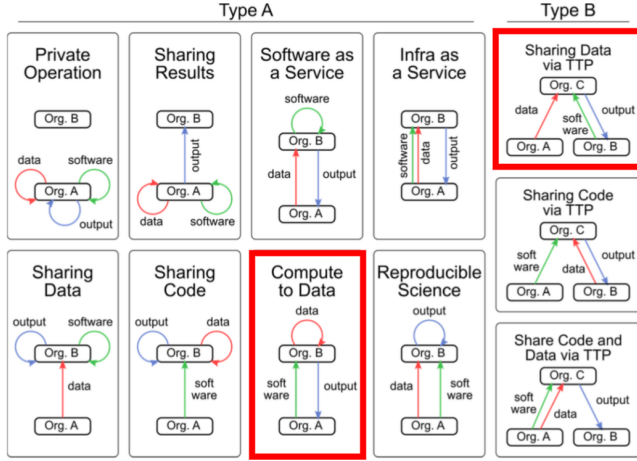


Figure 1: Data-sharing Archetypes [22]. The implemented archetypes in DYNAMOS are highlighted.

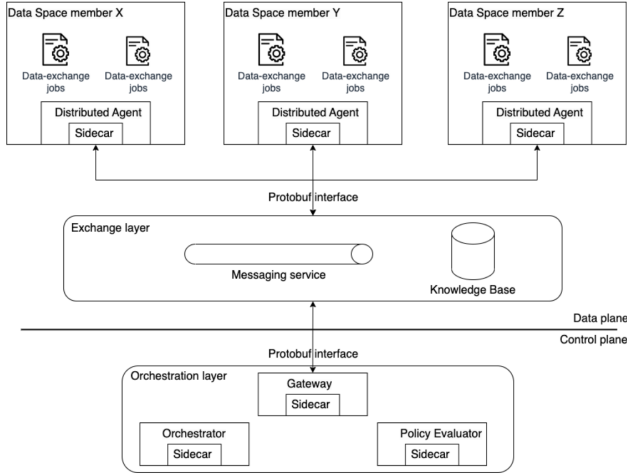


Figure 2: The architecture of the DYNAMOS system [25]

In DYNAMOS the archetypes are represented as ‘microservice chains’, which are sequences of microservices structured as directed acyclic graphs (DAGs). This representation ensures that each data exchange process follows a clear, non-redundant path from input to output. The chains are executed as ephemeral Kubernetes ‘jobs’ that are purposefully short lived to support policy-awareness.

Figure 2 gives an overview of the architecture of DYNAMOS. For example, when DYNAMOS receives a request, the *Gateway* service is responsible for the initial handling. Then the next step is performed by the *Policy Evaluator* that confirms whether the incoming request is valid given a preexisting set of policies. Then the *Orchestrator* dynamically selects the best archetype and generates the corresponding microservice chain. The microservice chain request is transmitted via the messaging queue to the *Distributed Agents* that are providing the data and the computational power. The ephemeral jobs are executed in the agents and the execution flow is defined by the microservice chain.

3 Energy efficiency pipeline

In this section, we describe the energy efficiency pipeline that starts with the use of Kubernetes-based telemetry tools (section 3.1) to retrieve energy consumption metrics, and subsequently the diagnostics methodology that uses Root Cause Analysis and Anomaly Detection to identify the main contributors to energy consumption.

3.1 Telemetry tools

The first step in the energy efficiency report pipeline involves collecting energy consumption metrics, that is quantitative measurements that reflect how much energy is used by system components over time. These metrics form the foundation for the investigation of the root causes of increased energy consumption.

The two most prominent tools for tracking energy consumption metrics are Scaphandre and Kepler. Both of them are open-source tools and both are compatible with Prometheus. Prometheus is used to collect container-level metrics such as CPU and memory utilization while Scaphandre and Kepler capture power consumption data and export it in a format compatible with Prometheus.

In this work, Kepler is chosen as the primary energy monitoring tool due to its superior accuracy in container-level energy metrics, which align closely with the objectives of our pipeline. The pipeline aims to detect inefficiencies by pinpointing specific containers or services within the system that are responsible for high energy consumption. As shown in the comparative study by Akbari et al. [2], Kepler outperforms Scaphandre in measuring container-level energy usage, making it particularly well-suited to Kubernetes-based microservice environments. In contrast, Scaphandre, while effective at collecting host-level metrics, tends to underestimate memory-related energy consumption at the container level, which is a limitation for detailed root cause analysis. Moreover, Centofanti et al. [9] and Gudepu et al. [14] both found that both tools gave similar results in their studies, suggesting a very comparable functionality.

In addition to its container-level precision, Kepler offers practical advantages in virtualized environments. Unlike Scaphandre, which depends on low-level hardware interfaces like powercap, Kepler supports energy estimation via pre-trained power models when direct access to hardware metrics is unavailable (Figure 3). This makes Kepler especially effective on cloud platforms that use virtual machines (VMs), where physical power sensors are often inaccessible [5]. In bare-metal Kubernetes clusters, Kepler can still use real-time metrics for greater accuracy.

3.2 Energy Efficiency Analysis

We introduce our energy efficiency analysis methodology and describe its operation. The methodology describes how to translate the raw power consumption measurements into actionable results by using analytical and statistical tools.

Central to the pipeline are two main components: the Anomaly Detection (AD) algorithm and the Root Cause Analysis (RCA) algorithm. Together, these algorithms identify and explain inefficiencies in energy consumption within the system. The AD algorithm flags data points that deviate from normal behavior, while the RCA algorithm investigates and explains the underlying root causes for these deviations. In microservice-based systems, such tools are essential for maintaining performance and optimizing energy use [12].

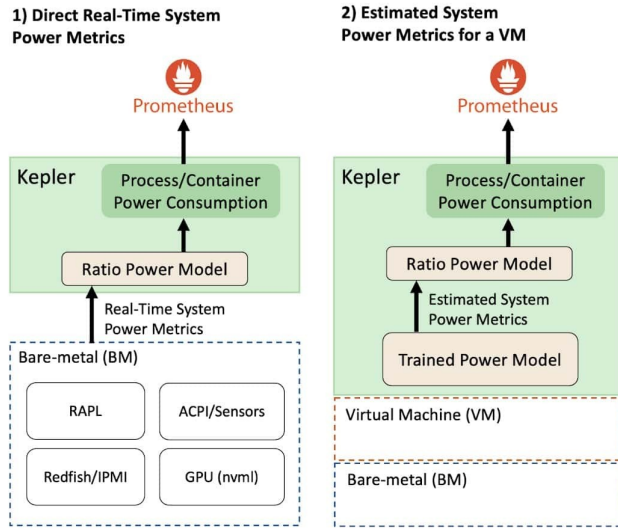


Figure 3: Bare Metal versus Virtual Machine for collecting Power consumption Metrics. (Obtained from Kepler Documentation [6])

In our pipeline, energy consumption data is monitored, and the AD algorithm detects irregularities by comparing observed behavior to expected patterns. When anomalies are identified, the RCA algorithm is triggered to determine the underlying causes of abnormal energy usage. The selection and implementation of these algorithms are informed by the study of Floroiu et al. [12], which evaluates various AD and RCA techniques specifically for energy consumption in microservices. Their evaluation is based on three key metrics: correctness (precision), completeness (recall), and accuracy (F-score). Specifically, they found that Balanced Iterative Reducing and Clustering Using Hierarchies [30] (BIRCH) on average performed the best out of all the AD algorithms, and Root Cause Discovery [16] (RCD) was the best performing RCA algorithm. Based on those findings, we choose BIRCH and RCD for our work. To ensure replicability and comparability, we use exactly the same algorithmic configuration as Floroiu et al. [12], including all parameters and threshold values.

To use the energy efficiency report pipeline described in Section 3.2, we train the RCA algorithm on a dataset representing normal system behavior—specifically, measurements taken when DYNAMOS was idle, outside of archetype execution. The anomaly detection algorithm (BIRCH) does not require training, but it benefits from preprocessing steps, as described by Floroiu et al. [12]. In our work, we follow the same preprocessing workflow before applying anomaly detection:

- (1) **Smoothing:** The data is smoothed using a rolling mean, which helps capture trends and reduces noise. In other words, data points are averaged to mitigate the impact of sudden outliers on different timestamps.
- (2) **Normalization:** The data is scaled to a common range to ensure comparability between values.

- (3) **Clustering with BIRCH:** A Clustering Feature (CF) tree is constructed by the BIRCH model.
- (4) **Anomaly Detection:** Anomalies are identified based on the distance of data points from their nearest cluster center. Points that exceed a set threshold distance are classified as anomalies.
- (5) **Results Handling:** The results are saved, with each data point labeled as either an *anomaly* or *normal*, based on the BIRCH clustering output.

4 Energy Efficiency Pipeline in DYNAMOS

In this section, we describe the application of the energy efficiency pipeline to DYNAMOS to diagnose the root causes of energy consumption. Subsequently, we analyze how the resulting insights guided the design and implementation of two optimizations aimed at reducing energy usage.

As a use case we use the Universities of The Netherlands scenario, first described by Stuttenheim et al. [25]. In this scenario there are two universities (UvA and VU) that each has a dataset about their employees, and the objective is to execute the SQL query (Listing 1) on their datasets. This can be achieved with the use of either the CtD or the DvTTP archetypes. In the former, the query is executed on the premises of the two universities, but in the latter the two universities share their datasets to a trusted third party (SURF) and the queries are executed there.

Listing 1: SQL Query for Selecting Distinct Records

```
SELECT DISTINCT p.Unieknr, p.Geslacht, [...]
FROM Personen p
JOIN Aanstellingen s ON p.Unieknr = s.Unieknr
LIMIT 20000;
```

In order to determine which are the root causes of the energy consumption of the archetypes we apply the methodology described in Section 3.2. The methodology requires first to collect the energy consumption measurements from the Kubernetes containers via Kepler and Prometheus. The measurements from Kepler and Prometheus were polled during 100 cycles of a sequence of seven DYNAMOS archetype requests each.

4.1 Root Cause Analysis

The RCA identified the *sidecar* and *system_processes* to be the main contributors to energy use. The *sidecar* container implements the sidecar design pattern as illustrated in Figure 2, and its role is to implement the communication processes within DYNAMOS. Thus, it is clear that the energy cost of the *sidecar* container is associated with the communication costs of the system. By contrast, the *system_processes* container aggregates a large and diverse set of system-level processes. This ambiguity stems from the Kubernetes' reporting model where energy metrics are aggregated at the container and pod level.

The initial results were useful in identifying high-level contributors, especially pinpointing the data transfer stage (via the *sidecar* container). However, in order to further examine the *system_processes* container we examined the execution time as a proxy for energy usage following the conclusion of Yuki and Rajopadhye [28], who argue that faster systems tend to be more energy-efficient due to reduced execution duration.

We measured the execution time per stage of the system (Startup, Data Processing, and Data Transfer stage) under the same configurations as previously described for the two archetypes. The Startup stage is the necessary time until the system can start executing the request, the Data Processing time refers to reading the data and executing the SQL query, and the Data Transfer refer to the time used for transferring the data necessary to execute the query according to each archetype.

Table 1: Latency per stage for the CtD and DvTTP archetypes

Archetype	Startup (%)	Processing Time (%)	Transfer Time (%)
CtD	36.06	43.12	20.82
DvTTP	29.47	27.77	42.76

Table 1 shows the percentage of execution time per stage for the two archetypes. For the archetype CtD the startup time and the processing time are the biggest contributors to the execution time. The transfer time is considerably lower but still is responsible for a significant percentage of delay. For the DvTTP archetype, notably, the transfer time is responsible for a much bigger delay in execution, which is expected as it involves moving the datasets to the Trusted Third Party (TTP) agent. However, startup and processing time still add up to a significant percentage of the delay. Overall, the tables show that all three operational stages contribute considerably to the execution time of both archetypes. All three of them are viable avenues for improving energy consumption for the two archetypes.

The results of the RCA along with the complementary execution time analysis reveal that all three stages contribute considerably to the execution time. Overall, we identify the following root causes of increased energy use:

- (1) **Startup Stage (RC1)**: The time required to initialize services, i.e. cold starts.
- (2) **Data Processing Stage (RC2)**: The time spent executing SQL queries and retrieving data.
- (3) **Data Transfer Stage (RC3)**: The time required for data transfers between services.

4.2 Optimizations

In this section, we demonstrate how the analysis results can inform actionable improvements. Based on the identified root causes, we implemented two optimizations designed to reduce energy consumption. The first optimization introduced in DYNAMOS aims to make the execution of archetypes more efficient through caching, and the second with the use of data compression.

4.2.1 Request Caching. Caching is a well-established system design strategy that involves storing frequently accessed data in a location that is faster and easier to retrieve. Its primary goal is to improve the performance and efficiency of the system by reducing the time and resources required to access commonly used data. Studies have demonstrated that caching can significantly improve energy efficiency [19, 27].

This optimization approach addresses all three root causes: it improves Startup (**RC1**) by avoiding the creation of new jobs for identical requests, thus bypassing service initialization; it reduces

Data Processing (**RC2**) costs by eliminating redundant computations; and it mitigates Data Transferring (**RC3**) overhead, since cached results can be reused without additional data exchanges.

The two main in-memory caching tools we consider are Memcached⁸ and Redis⁹. We choose Redis because it provides support for more data structures, it allows for bigger key values (2GB compared to 250 Bytes) and has a wider set of capabilities.

We implement caching in DYNAMOS at each Distributed Agent (Figure 2) to preserve data sovereignty of each data owner. In this design, each agent is responsible for updating and maintaining each local data cache according to incoming requests. Moreover, this approach ensures that caching does not interfere with the Policy Evaluator service, allowing the system to adapt to policy changes. The caching mechanism registers a new entry when no cache hit occurs and applies time-to-live (TTL) of 10 minutes to prevent stale or unused entries from persisting. While the optimal caching parameters depend on the specific expected usage, we find that the chosen configuration performs effectively for our use case.

4.2.2 Transferred Data Compression. Data compression, the process of storing information in a compact form, can significantly reduce energy consumption. This is because the energy required for pre-processing tasks, such as encoding, is in most cases negligible compared to the energy consumed during data broadcasting [23]. The primary purpose of compression is to minimize storage and bandwidth requirements for data transmission, which inherently leads to energy savings. By reducing the size of data transferred over networks or stored on disk, compression decreases the resources and energy required for these operations [3, 15].

In the context of archetype execution in DYNAMOS, compression is another optimization that holds the potential for significant energy savings, specifically by addressing the *Data Transfer* root cause (**RC3**). By compressing the data transferred between components, can considerably reduce the size of the data being transmitted. A smaller data size could lead to a significant reduction in the time required for data transmission, improving overall energy efficiency.

There are multiple widely used open-source compression algorithms (e.g. Gzip, Brotli, Zstandard, Deflate). We select the Gzip¹⁰ lossless compression algorithm due to its great balance between compression ratio and speed [26]. While higher compression rates help reduce transmission costs, compression speed is also important, as slower performance can increase overall energy consumption. Gzip is reportedly the most widely used compression algorithm [18], with broad compatibility and near-universal support.

DYNAMOS employs three different protocols for its communication tasks, *HTTPS* mainly for external requests, *AMQ* mainly for metadata and system control, and *gRPC* mainly for data transfer between microservices. We focus on the communication via the *AMQ* and *gRPC* protocols, which typically have bigger loads and stand to improve the most from compression. We introduce a compression and decompression stage before and after data transfers, respectively. Additionally, compression was applied conditionally based on payload size, i.e. for small data loads, compression yields

⁸www.memcached.org/

⁹redis.io/solutions/caching/

¹⁰www.gzip.org/

minimal benefits and may introduce unnecessary overhead. Therefore, we enabled compression only when the payload size exceeded 100 bytes. In practice, payloads exceeding this threshold correspond to dataset transfers and SQL query results, which are stored locally in the memory of the distributed agents without prior compression. This setup focuses compression efforts on data exchanges where it has the greatest impact, while avoiding unnecessary processing of smaller messages.

5 Experiments

In this section, we compare the original data exchange archetypes with their optimized counterparts in both local and distributed environments. We begin with a short description of the local and the distributed setup, and then we move to the evaluation of the optimized archetypes in terms of energy consumption. Extensive deployment guidelines can be found in our GitHub repository¹¹.

The local deployment of DYNAMOS was on a system with 32GB RAM and 12 CPU cores. The configuration of DYNAMOS is as described in the original work [25] with the distributed agents that correspond to the Universities of the Netherlands use case. Specifically, it includes the universities *UvA* and *VU* and the trusted third party *SURF* as distributed agents.

5.1 Distributed deployment on FABRIC

The distributed deployment of DYNAMOS is implemented on the FABRIC testbed. FABRIC consists of a network on several sites in USA, Europe and Asia, each equipped with substantial computational and storage resources. These sites are interconnected by high-speed, dedicated optical links, facilitating seamless data transfer and collaboration.

FABRIC's highly customizable, geographically distributed infrastructure provides an ideal platform for evaluating the energy consumption of data exchange archetypes[10]. It allows for consistent experimentation across different distributed setups within a unified environment. Overall, the advanced capabilities, the geographic distribution, and the customizability make FABRIC a suitable testbed for studying energy consumption in DYNAMOS.

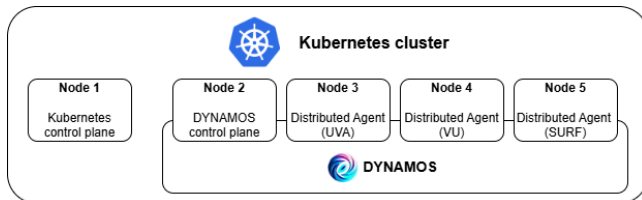


Figure 4: Structure of the Kubernetes cluster on FABRIC

For the deployment, we set up a single-cluster Kubernetes environment on FABRIC consisting of one control plane node and multiple worker nodes (Figure 4). The Kubernetes control plane maintains the cluster state, manages workloads, and orchestrates container deployment, while the worker nodes execute application containers and handle data exchange and computational tasks. Kubernetes control plane is deployed on a Virtual Machine (VM)

¹¹github.com/CollinPoetoehena/EnergyEfficiency_DYNAMOS

with 4CPU cores and 16GB RAM, while each of the other nodes on a VM with 8 CPU cores and 16GB RAM. Kubernetes is bootstrapped using Kubeadm¹², the official setup tool maintained by the Kubernetes project. Alternative tools such as Kops and Kubespray were considered, however, Kubeadm is chosen for its flexibility, fine-grained control, and comprehensive documentation, which makes it a strong choice for custom deployments [8].

5.2 Evaluation

We evaluate the optimized energy-efficient data exchange archetypes described in Sec. 4.2. The energy measurements are obtained using the Kepler and Prometheus telemetry tools described in Sec. 3.1. Initially we compare the optimized archetypes to the original in the local and distributed (FABRIC) installation, and then we study the correlation between execution times and energy consumption.

Prior to analysis, unsuccessful executions and anomalous measurements are excluded using the DBSCAN algorithm, following the same configuration as Koedijk and Oprescu [17]. Normality of the remaining data is assessed using the Shapiro–Wilk test, and it indicates non-normal distributions. Consequently, statistical significance between baseline and optimized configurations is evaluated using the non-parametric Mann–Whitney U test with a significance level of 0.05 (alpha). Full analysis is available in [20].

5.2.1 Energy Efficient Optimizations Evaluation. Table 2 shows the results of the execution for the **Compute to Data Archetype**. The data compression optimization demonstrates some improvement compared to baseline for both energy and execution duration but it is not statistically significant in the local setup. On the FABRIC environment, the compression offers a statistically significant improvement on the execution duration. Interestingly, the duration improvement is very minor, but due to the highly consistent environment of FABRIC it is statistically significant. The caching optimization offers statistically significant better performance over the baseline in both energy and time cost. In both the local and FABRIC setup the caching optimization minimizes the cost to a fraction of the baseline.

Table 2: Energy-efficient *Compute to Data Archetype* execution. Positive statistical significance is marked with *.

		Energy (J)		Duration (s)	
		Mean	StD	Mean	StD
Local Setup	Baseline	1003.871	249.825	7.598	1.532
	Compression	928.958	125.386	7.324	1.105
	Caching	13.153*	69.026	0.397*	0.487
FABRIC	Baseline	2083.912	543.171	6.537	0.158
	Compression	1903.897	414.278	6.247*	0.094
	Caching	101.575*	245.735	0.347*	0.440

Table 3 shows the comparison of the different versions of the **Data via TTP Archetype**, executed on the local and the FABRIC environment. In this case, the compression optimization shows more mixed results, as it displays positive results only on the time cost, but negative results on the energy costs, in both local and

¹²kubernetes.io/docs/reference/setup-tools/kubeadm/

FABRIC setup. Specifically, the time improvement (on FABRIC), and the increase of the energy cost (on local setup) are statistically significant. The caching optimization again demonstrates statistically significant improvements in both time and energy cost and on both environments. Although the caching is very impactful, it is less efficient in the FABRIC environment, compared to the local installation.

Table 3: Energy-efficient Data via TTP Archetype execution. Positive statistical significance is marked with *, negative with †.

		Energy (J)		Duration (s)	
		Mean	StD	Mean	StD
Local Setup	Baseline	1174.316	172.401	8.327	0.718
	Compression	1348.404†	236.661	8.592	1.053
	Caching	39.120*	92.453	0.582*	0.513
FABRIC	Baseline	4071.103	567.294	5.357	0.319
	Compression	4079.854	592.702	5.258*	0.215
	Caching	179.234*	319.115	0.349*	0.378

Overall, the experiments demonstrate that the data transfer compression optimization leads to modest improvements across most measurements. However, in the case of the DvTTP archetype, performance is negatively impacted. In contrast, the caching optimization consistently delivers significant benefits in all tested scenarios, suggesting it can be a highly effective strategy for reducing both energy and time costs. Comparing the two environments, FABRIC exhibits higher energy consumption but lower execution times. The increased energy usage may be attributed to the more complex configuration of DYNAMOS on FABRIC, highlighting potential areas for setup optimization and reflecting the inherent overhead of distributed systems. Meanwhile, the faster execution times on FABRIC underscore its superior computational capabilities compared to the local setup.

5.2.2 Correlation between energy and execution time. This section explores the relationship between execution time and energy consumption across all experiments. While the previous section focused on analyzing optimizations individually, here we examine whether execution time can reliably predict energy efficiency by performing correlation analysis for both the local and FABRIC environments.

Figure 5 illustrates the correlation between energy consumption and execution time across all individual experiments on the local setup. The results indicate a moderate positive correlation between these two metrics, with a Kendall tau coefficient of $\tau = 0.553$. This suggests that, in general, longer execution times tend to be associated with higher energy usage. However, the scattering of the data point also shows that this relationship is not perfectly linear, and shorter execution times do not always result in the lowest energy consumption for every individual experiment run.

Figure 6 visualizes the relationship between energy consumption and execution time for all implementations on the FABRIC setup. In contrast to Experiment 1, the correlation between energy and execution time is noticeably lower, with a Kendall tau value of $\tau = 0.222$. This indicates a low positive correlation: while there is a general tendency for longer execution times to be associated with

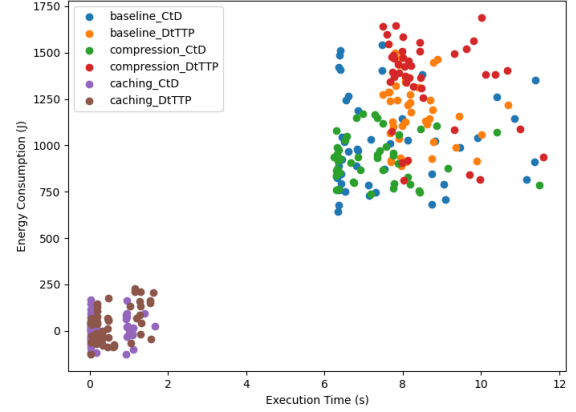


Figure 5: Execution time vs Energy consumption for the local deployment

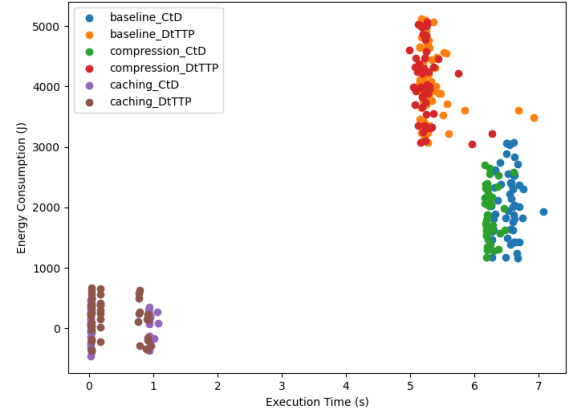


Figure 6: Execution time vs Energy consumption for the FABRIC deployment

higher energy consumption, the relationship is less pronounced and more variable in the FABRIC environment.

As shown in Figure 6, caching implementations cluster at the lower-left part of the plot, reflecting both low execution times and energy consumption. Similar to the local environment, a few caching experiments show slightly elevated mean execution times. Other configurations, particularly the baseline and compression implementations, are more dispersed. This demonstrates that faster execution time does not always imply lower energy use for every individual run. Interestingly, when comparing the results between the local and the FABRIC environments, the FABRIC results are clearly more stable in terms of execution time, likely a result of the stability of the FABRIC infrastructure.

6 Conclusion

In this work, we presented a green computing application on the DYNAMOS middleware for data exchange archetypes, covering the entire process from the energy consumption telemetry and diagnostic analysis to the implementation and evaluation of the optimized archetypes. Specifically, we presented the energy consumption monitoring pipeline using Kepler and Prometheus that is suitable for the container-based architecture of DYNAMOS. We then applied Root Cause Analysis to the collected energy metrics to identify the primary contributors to energy consumption. Based on these findings, we implemented two optimizations targeting the root causes of the identified root causes, namely, a caching mechanism and data compression. We evaluated the optimized archetypes both in a local setup and in a distributed setup on the FABRIC testbed. Our work demonstrates how complex and distributed container-based systems can effectively monitor and reduce energy consumption. Furthermore, our results indicate that caching and data transfer compression can provide significant benefits for data exchange archetypes, depending on the specific use case.

Potential future work directions include additional optimizations for data exchange archetypes, incorporating the energy efficiency criterion into the DYNAMOS archetype selection process, validating energy estimates obtained on the deployed pipeline versus actually measured energy consumption at machine level, generalizing our insights by contrasting our approach to broader state-of-the-art.

Acknowledgments

This work was partially supported by the project: “BNVw: Budget-neutral sustainable home improvements (MOOI-224049)”, and by GreenDIGIT, an European Union project funded under the grant agreement 101131207.

References

- [1] Vedangi Agarwal, Konark Sharma, and Abha Kiran Rajpoot. 2021. A Review: Evolution of Technology towards Green IT. In *2021 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*. 940–946.
- [2] Milad Akbari, Raffaele Bolla, Roberto Bruschi, Franco Davoli, Chiara Lombardo, and Beatrice Siccardi. 2024. A Monitoring, Observability and Analytics Framework to Improve the Sustainability of 5G Technologies. In *2024 IEEE International Conference on Communications Workshops (ICC Workshops)*. 969–975. <https://doi.org/10.1109/ICCWorshops59551.2024.10615948>
- [3] Halah Mohammed Al-Kadhim and Hamed S. Al-Raweshidy. 2021. Energy Efficient Data Compression in Cloud Based IoT. *IEEE Sensors Journal* 21, 10 (2021).
- [4] Marcelo Amaral, Huamin Chen, Tatsuihiro Chiba, Rina Nakazawa, Sunyanan Choochothaew, Eun Kyung Lee, and Tamar Eilam. 2023. Kepler: A Framework to Calculate the Energy Consumption of Containerized Applications. In *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*. 69–71.
- [5] Marcelo Amaral, Huamin Chen, Tatsuihiro Chiba, Rina Nakazawa, Sunyanan Choochothaew, Eun Kyung Lee, and Tamar Eilam. 2024. Process-Based Efficient Power Level Exporter. In *2024 IEEE 17th International Conference on Cloud Computing (CLOUD)*. 456–467. <https://doi.org/10.1109/CLOUD62652.2024.00058>
- [6] Marcelo Amaral, Sunyanan Choochothaew, Eun Kyung Lee, Huamin Chen, and Tamar Eilam. 2023. Exploring Kepler’s potentials: unveiling cloud application power consumption. <https://www.cncf.io/blog/2023/10/11/exploring-keplers-potentials-unveiling-cloud-application-power-consumption> Accessed: 16.01.2026.
- [7] Jose Balanza-Martinez, Patricia Lago, and Roberto Verdecchia. 2024. Tactics for Software Energy Efficiency: A Review. In *Advances and New Trends in Environmental Informatics 2023*, Volker Wohlgemuth, Dieter Kranzlmüller, and Maximilian Höb (Eds.). Springer Nature Switzerland, Cham, 115–140.
- [8] Lincoln Bryant, Robert W Gardner, Fengping Hu, David Jordan, and Ryan P Taylor. 2024. Kubernetes Deployment Options for On-Prem Clusters. *arXiv preprint arXiv:2407.01620* (2024).
- [9] Carlo Centofanti, José Santos, Venkateswarlu Gudepu, and Koteswararao Kondepudi. 2024. Impact of power consumption in containerized clouds: A comprehensive analysis of open-source power measurement tools. *Computer Networks* 245 (2024), 110371.
- [10] Anestis Dalgkitsis, Alexandros Koufakis, Jorrit Stutterheim, Aleandro Mifsud, Priyanka Atwani, Leon Gommans, Cees de Laat, Chrysa Papagianni, and Ana Opreescu. 2024. Secure Collaborative Model Training with Dynamic Federated Learning in Multi-Domain Environments. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 755–759.
- [11] João Daniel, Eduardo Guerra, Thatiane Rosa, and Alfredo Goldman. 2023. Towards the Detection of Microservice Patterns Based on Metrics. In *2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. 132–139. <https://doi.org/10.1109/SEAA60479.2023.00029>
- [12] Maximilian Stefan Floroiu, Stefano Russo, Luca Giamattei, Antonio Guerriero, Ivano Malavolta, and Roberto Pietrantuono. 2024. Anomaly Detection and Root Cause Analysis of Microservices Energy Consumption. (2024).
- [13] Charlotte Freitag, Mike Berners-Lee, Kelly Widdicks, Bran Knowles, Gordon S. Blair, and Adrian Friday. 2021. The real climate and transformative impact of ICT: A critique of estimates, trends, and regulations. *Patterns* 2, 9 (2021), 100340. <https://doi.org/10.1016/j.patter.2021.100340>
- [14] Venkateswarlu Gudepu, Rajashekhar Reddy Tella, Carlo Centofanti, José Santos, Andrea Marotta, and Koteswararao Kondepudi. 2024. Demonstrating the Energy Consumption of Radio Access Networks in Container Clouds. In *the IEEE/IFIP Network Operations and Management Symposium (NOMS)*.
- [15] Mahdih Hajiloo Vakili and Zahra Shirmohammadi. 2024. EDC-ER: An Efficient Data Compression Method for Energy Reduction in WBANs. *IEEE Access* 12 (2024), 155274–155286. <https://doi.org/10.1109/ACCESS.2024.3476424>
- [16] Azam Ikram, Sarthak Chakraborty, Subrata Mitra, Shiv Saini, Saurabh Bagchi, and Murat Kocaoglu. 2022. Root Cause Analysis of Failures in Microservices through Causal Discovery. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc.
- [17] Lukas Koedijk and Ana Opreescu. 2022. Finding Significant Differences in the Energy Consumption when Comparing Programming Languages and Programs. In *2022 International Conference on ICT for Sustainability (ICT4S)*. 1–12.
- [18] Kirill Kryukov, Mahoko Takahashi Ueda, So Nakagawa, and Tadashi Imanishi. 2020. Sequence Compression Benchmark (SCB) database—A comprehensive evaluation of reference-free compressors for FASTA-formatted sequences. *Giga-Science* 9, 7 (07 2020), gaa072.
- [19] Lubna B Mohammed, Alagan Anpalagan, and Muhammad Jaseemuddin. 2023. Energy and latency efficient caching in mobile edge networks: Survey, solutions, and challenges. *Wireless Personal Communications* 129, 2 (2023), 1249–1283.
- [20] Collin Poetoehena. 2025. *Energy Efficient Implementation of Data Exchange Archetypes in DYNAMOS*. Master’s thesis. University of Amsterdam, Amsterdam, The Netherlands. https://scripties.uba.uva.nl/search?id=record_55890
- [21] Yamina Romani, Okba Tibermacine, and Chouki Tibermacine. 2022. Towards Migrating Legacy Software Systems to Microservice-based Architectures: a Data-Centric Process for Microservice Identification. In *2022 IEEE 19th International Conference on Software Architecture Companion (ICSA-C)*.
- [22] Sara Shakeri, Lourens Veen, and Paola Grosso. 2020. Evaluation of Container Overlays for Secure Data Sharing. In *2020 IEEE 45th LCN Symposium on Emerging Topics in Networking (LCN Symposium)*. 99–108.
- [23] Biljana Risteska Stojkoska and Zoran Nikolovski. 2017. Data compression for energy efficient IoT solutions. In *2017 25th Telecommunication Forum (TELFOR)*.
- [24] Jorrit Stutterheim. 2023. *DYNAMOS: Dynamically Adaptive Microservice-based OS. A Middleware for Data Exchange Systems*. Master’s thesis. University of Amsterdam, The Netherlands. scripties.uba.uva.nl/search?id=record_53890
- [25] Jorrit Stutterheim, Aleandro Mifsud, and Ana Opreescu. 2024. DYNAMOS: Dynamic Microservice Composition for Data-Exchange Systems, Lessons Learned. In *International Conference on Software Architecture (ICSA)*.
- [26] Z Syed and T Soomro. 2018. Compression Algorithms: Brotli, Gzip and Zopfli Perspective. *Indian Journal of Science and Technology* 11, 45 (2018), 1–4.
- [27] Huiqing Xu, Hengjie Huang, and Gaocai Wang. 2016. An Energy-Efficient Caching Strategy Based on Coordinated Caching for Green Content-Centric Network. In *Security, Privacy and Anonymity in Computation, Communication and Storage*, Guojun Wang, Indrakshi Ray, Jose M. Alcaraz Calero, and Sabu M. Thampi (Eds.). Springer International Publishing, Cham, 168–175.
- [28] Tomofumi Yuki and Sanjay Rajopadhye. 2013. Folklore Confirmed: Compiling for Speed Compiling for Energy. In *International Workshop on Languages and Compilers for Parallel Computing*. Springer, 169–184.
- [29] Lu Zhang, Reginald Cushing, Leon Gommans, Cees De Laat, and Paola Grosso. 2019. Modeling of collaboration archetypes in digital market places. *IEEE Access* 7 (2019).
- [30] Tian Zhang, Raghu Ramakrishnan, and Miron Livny. 1996. BIRCH: an efficient data clustering method for very large databases. *SIGMOD Rec.* 25, 2 (6 1996).